

Code Review — layer business (Kong + realm rsk)

Repo `nimigeanconstantinon/ms-gitops` · branch `master` · 2026-07-22 · read-only, codul nu a fost modificat

Sumar

Infra e matură; findings-urile din review-ul precedent (2026-07-07) sunt **rezolvate**. Migrarea a avansat cu Kong + clientul swagger dedicat. Un singur bug nou blochează sync-ul Kong; restul e igienă.

#	Sev	file:line	Problemă
B1		<code>argo-apps/app-kong.yaml:20</code>	<code>valueFiles</code> pointează la un path fără <code>rsk/</code> → Kong nu se sincronizează
M1		<code>business/rsk/keycloak/realm-rsk.yaml:95</code>	Secret client confidențial în clar, comis în git (<code>spring-api</code>)
C1		<code>bootstrap/root.yaml:14</code>	Comentariu leftover din starter
C2		<code>business/charts/microservice/templates/</code>	Chart fără <code>_helpers.tpl</code> / label-uri standard / <code>NOTES.txt</code>

B1 — Kong: valueFiles pe cale inexistentă

CRITICAL

Kong rămâne OutOfSync — value file negăsit

argo-apps/app-kong.yaml:20

Cele 3 source-uri de repo din `Application` Kong au fost mutate sub `business/rsk/...`, dar `valueFiles` a rămas pe structura veche. Verificat cu `ls`: fișierul din manifest nu există.

BEFORE → AFTER

Linie	Before	After
:20	<code>\$values/business/data-service/kong/values.yaml</code> 	<code>\$values/business/rsk/data-service/kong/values.yaml</code>

DOVADA

```
# app-kong.yaml – ingress si declarative au deja rsk/, doar valueFiles nu:
:20 - $values/business/data-service/kong/values.yaml
:28 path: business/rsk/data-service/kong/ingress
:32 path: business/rsk/data-service/kong/declarative
```

Mecanism: cu multi-source `$values`, ArgoCD rezolvă calea din source-ul cu `ref: values`. Fișierul lipsă → `ComparisonError`, chart-ul Kong nu primește deloc values → `Application Unknown/OutOfSync`. Aliniind linia :20 cu celelalte două, se deblochează.

M1 — spring-api: secret în clar

🟡 IMPORTANT Parolă de client confidențial comisă în repo

business/rsk/keycloak/realm-rsk.yaml:95

```
:90 - clientId: spring-api
:93   publicClient: false
:95   secret: "your-secret-key-here-change-in-production"
```

La import, Keycloak ia secretul *literal* — devine parola reală a clientului, vizibilă în istoricul git.

Opțiune	Când
A. Scoate clientul (recomandat)	<code>spring-api</code> nu e consumat — <code>data-service</code> merge pe <code>data-service-swagger</code> (public)
B. <code>SealedSecret</code> + referință	Dacă backend-ul chiar are nevoie de client confidențial — pattern-ul de pe car-platform

De ce: un secret comis rămâne în istoric și după ștergere. Regula clusterului — materialul sensibil trece prin sealed-secrets, nu în clar în repo.

🟢 Cleanups

C1 — leftover starter · `bootstrap/root.yaml:14`

```
:14 targetRevision: master # SAU master - ajusteaza dupa branch-ul real
```

Comentariu-template fără sens (branch-ul e clar `master`). Șterge coada comentariului.

C2 — igiena chart `microservice` · `templates/`

Doar `deployment.yaml`, `service.yaml`, `ingress.yaml`. Lipsesc:

- `_helpers.tpl` → nume + label-uri duplicate în fiecare template.
- Label-uri standard `app.kubernetes.io/name`, `/managed-by` — le folosesc Prometheus/Kibana la corelare.
- `NOTES.txt`. Fără impact funcțional — de curățat înainte de a reutiliza chart-ul pe al 2-lea serviciu + UI.

✅ Rezolvate din review-ul precedent (2026-07-07)

Vechi	Stare
F1 — <code>data-service</code> definit de 2 ori	✅ a rămas doar <code>app-data-service.yaml</code>
Wildcard <code>*</code> în <code>redirectUri</code> s	✅ scos — <code>redirect-uri</code> exacte; <code>"+"</code> doar în <code>webOrigins</code>
PKCE pe swagger	✅ S256

Ce e bine (de păstrat)

- Multi-source `$values` corect pe `data-service` (chart + values, `ref: values`).
- Tag imutabil `06e667d` pus de CI, fără `:latest` în cluster.
- `secretEnv` din secret `data-service-db` — zero secrete în values.
- Realm: `redirect-uri` exacte, PKCE `S256`, `directAccessGrants` doar în dev.
- `resources`: request CPU fără limit CPU, limit doar pe memorie — pattern corect.

Q&A — de verificat de tine

1. **Kong sync:** după fix-ul B1 și push, ce întoarce `kubectl -n argocd get app kong -o jsonpath='{.status.sync.status}'` ? Dacă rămâne `OutOfSync` , ce zice tab-ul *Manifest* în UI?
2. **spring-api:** mai are ceva nevoie de acest client, sau `data-service-swagger` (public) acoperă tot fluxul de login pe Swagger? Dacă nimic nu-l consumă, de ce să-l ții în realm?
3. **Kong vs nginx:** requestul spre `data.icode.mywire.org` intră prin nginx Ingress al chart-ului (`ingress.enabled: true`) SAU prin gateway-ul Kong? Cine termină TLS-ul și cine face rutarea — nu cumva le ai pe amândouă în paralel?

Prioritate: **B1** întâi (deblochează Kong) → **M1** → C1/C2. Review read-only; before/after sunt în document, nu aplicate în cod. Stop & „next”.